

Testing Foundations (2/2)

SE 413/513 — Software Testing
Grand Valley State University

Welcome Back

Last time: why testing matters, the error → fault → failure chain, verification vs. validation, and the seven principles.

Key Idea

Testing builds justified confidence; it does not build proof.

Today: how real teams *organize* that discipline — classifying and tracking defects, why cost grows with delay, the levels of testing, and the V-model/STLC that plan testing alongside development.

Learning Objectives — Lecture 2

By the end of today, you should be able to:

- Classify defects by **severity** and **priority**, and describe a typical defect life cycle.
- Explain why the cost of fixing a defect grows the later it is discovered.
- Describe the levels of testing (unit, integration, system, acceptance) and how they relate to the V-model.
- Describe the phases of the Software Testing Life Cycle (STLC).
- Recognize common misconceptions about what testing can and cannot guarantee.

Roadmap

- 1 Defects: Classification & Life Cycle
- 2 The Cost of Defects
- 3 Levels of Testing
- 4 The V-Model and STLC
- 5 Common Misconceptions
- 6 Wrap-Up

Severity vs. Priority

Once a failure is observed, it needs to be reported, tracked, and resolved. These two attributes are frequently confused, but answer different questions.

Severity: the *technical impact* of a defect — how badly does it break functionality, data integrity, or safety?

Priority: the *urgency* of fixing it — how soon, relative to other work, should this be addressed?

Severity/Priority Levels

Level	Example (Severity)	Example (Priority)
??	System crashes; data loss; security breach; safety hazard	??
??	Major feature broken, no workaround	??
??	Feature works, but a usability problem	??
??	Cosmetic issue, typo	??

Severity/Priority Levels

Level	Example (Severity)	Example (Priority)
Critical/Urgent	System crashes; data loss; security breach; safety hazard	Must be fixed before the next release
High	Major feature broken, no workaround	Should be fixed this sprint
Medium	Feature works, but a usability problem	Scheduled for an upcoming release
Low/Trivial	Cosmetic issue, typo	Fixed when convenient, or deferred

Severity and Priority Can Diverge

A defect can be:

High severity, low priority — a crash in a rarely used admin tool that has an easy workaround.

Low severity, high priority — a misspelled company name on the homepage: functionally harmless, but embarrassing enough to fix immediately.

Key Idea

Severity asks “how bad, *if it happens?*” Priority asks “how soon, *given everything else we need to do?*” Always assess them separately.

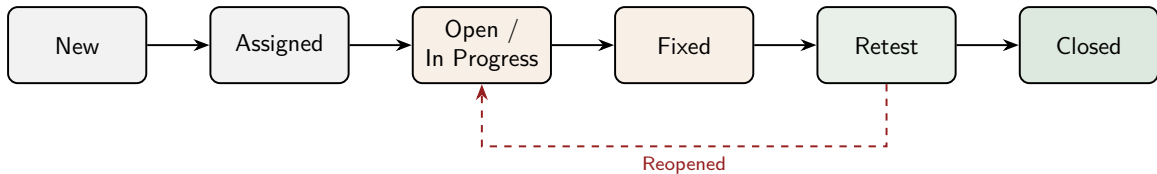
Discussion Prompt

Discussion Prompt

Assign a severity (Critical/High/Medium/Low) *and* a priority to each, and justify:

1. A checkout page crashes for all users on the busiest shopping day of the year.
2. An internal admin report, used once a quarter, shows a date in the wrong format.

The Defect Life Cycle



A tester (often *not* the fixer) verifies the fix at **Retest**. If the failure is still present — or a new one appeared — it goes back to **Open**.

Tip

Tip

A defect report typically records: a unique ID, a short summary, environment/version, precise **steps to reproduce**, expected result, actual result, severity, priority, and status.

When you file a defect, imagine you will never get to talk to the reporter again. Could someone else reproduce the failure using only what you wrote down?

Roadmap

- 1 Defects: Classification & Life Cycle
- 2 The Cost of Defects**
- 3 Levels of Testing
- 4 The V-Model and STLC
- 5 Common Misconceptions
- 6 Wrap-Up

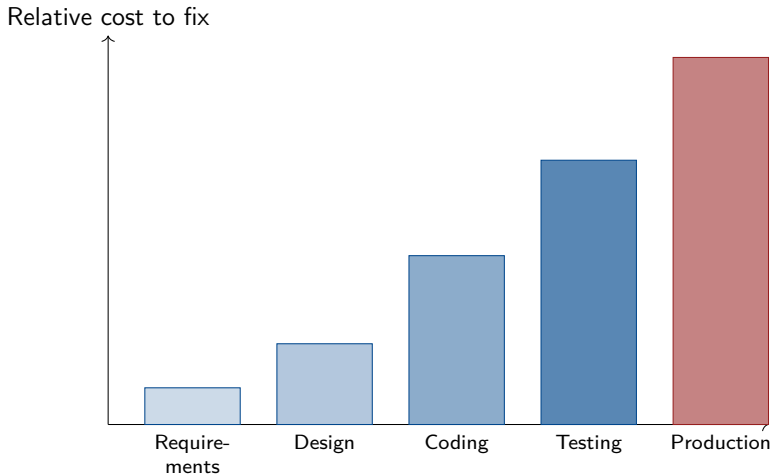
The Cost of Fixing a Defect Grows With Delay

Key Idea

One of the most consistently reproduced findings in software engineering research: **the cost of fixing a defect increases the later it is discovered** in the development lifecycle.

- A defect caught during requirements review might cost the time to correct a document.
- The same conceptual mistake, surviving to a production incident, can cost emergency patches, outages, lost revenue, and reputational damage no patch can fully undo.

Illustrating the Growth



Illustrative only – not to scale; exact multipliers vary by study and project.

Connecting Back to Lecture 1

Neither Knight Capital nor Mars Climate Orbiter lost money because a single line of code was “wrong” in some abstract sense.

They lost money because a defect that *could have been caught* during design review, code review, or staged deployment testing instead reached production — where the cost of the same underlying mistake was orders of magnitude higher.

Key Idea

This is the economic argument for **shift-left testing** (Principle 3): every phase you push a defect’s discovery earlier into tends to reduce the cost of fixing it, often dramatically.

Discussion Prompt

Discussion Prompt

Why does the cost of a defect tend to rise the later it is discovered? What kinds of costs are involved *besides* engineering time?

Roadmap

- 1 Defects: Classification & Life Cycle
- 2 The Cost of Defects
- 3 Levels of Testing**
- 4 The V-Model and STLC
- 5 Common Misconceptions
- 6 Wrap-Up

Four Levels, Increasing Scope

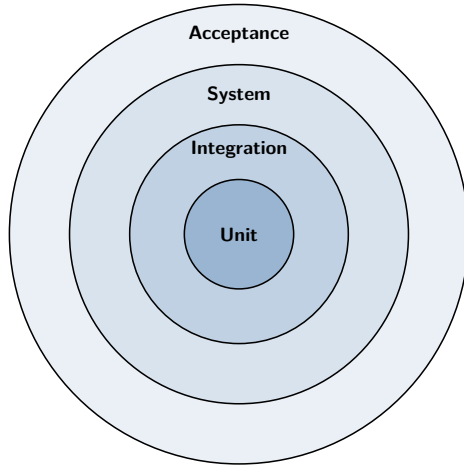
Unit testing: a single unit of code (function, method, class) in isolation, often replacing dependencies with test doubles.

Integration testing: how multiple units work together, focusing on the *interfaces* between them — exactly where the Mars Climate Orbiter defect lived.

System testing: the complete, integrated system against its overall requirements.

Acceptance testing: validates the system against user/business needs, often deciding release-readiness.

Levels of Testing: Increasing Scope



A Unit Test, Revisited

Recall `calculate_late_fee()` from Lecture 1. A minimal unit test, using Python's `unittest`:

```
class TestLateFee(unittest.TestCase):
    def test_typical_fee(self):
        self.assertEqual(calculate_late_fee(3), 4.5)

    def test_negative_days_should_not_be_negative_fee(self):
        # Documents the requirement; currently FAILS, exposing
        # the fault from Lecture 1.
        self.assertGreaterEqual(calculate_late_fee(-4), 0)
```

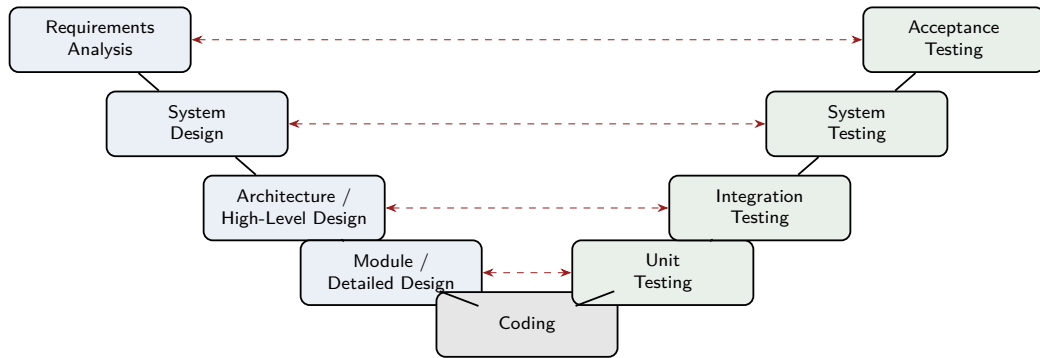
Tip

A failing test that accurately captures a requirement is not a problem to hide — it is exactly what Principle 1 describes: it shows the presence of a defect.

Roadmap

- 1 Defects: Classification & Life Cycle
- 2 The Cost of Defects
- 3 Levels of Testing
- 4 The V-Model and STLC**
- 5 Common Misconceptions
- 6 Wrap-Up

The V-Model

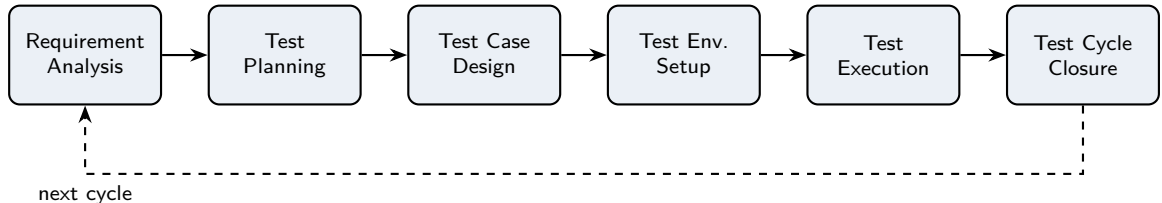


Each development activity (left) is paired with a testing activity (right) that verifies and validates it — testing is **planned alongside** development, not bolted on after.

The V-model implies that **acceptance test cases can be drafted as soon as requirements are written** — long before any code exists — because acceptance tests validate the *requirements*, not the implementation.

The Software Testing Life Cycle (STLC)

The STLC describes the phases testing itself goes through, **in parallel with** (not strictly after) development.



1. **Requirement analysis** — identify what's testable; catch ambiguity early (Principle 3).
2. **Test planning** — scope, approach, resources, entry/exit criteria.
3. **Test case design** — write detailed test cases and test data.
4. **Test environment setup** — prepare where tests will run.
5. **Test execution** — run tests, log results, file defect reports.
6. **Test cycle closure** — evaluate against exit criteria, document lessons learned.

Roadmap

- 1 Defects: Classification & Life Cycle
- 2 The Cost of Defects
- 3 Levels of Testing
- 4 The V-Model and STLC
- 5 Common Misconceptions**
- 6 Wrap-Up

Common Pitfall

Common Pitfall

“100% code coverage means the code is bug-free.” Coverage measures which lines were *executed* during testing, not whether the correct *outcomes* were checked.

A test can execute every line of `calculate_late_fee()` while never asserting anything about negative input — and coverage tools will happily report 100%.

Common Pitfall

Common Pitfall

“Testing is QA’s job, not mine.” Modern software engineering treats quality as a **shared responsibility**.

Developers write unit tests and participate in code review; QA engineers design broader test strategies; product stakeholders participate in acceptance testing. No single role can single-handedly guarantee quality.

Common Pitfall

Common Pitfall

“If it passed all our tests, it’s correct.” Passing tests demonstrates the *absence of the specific failures you checked for* — not the presence of correctness in any absolute sense.

This is Principles 1 and 7, and the Ariane 5 and Mars Climate Orbiter case studies, all over again.

Roadmap

- 1 Defects: Classification & Life Cycle
- 2 The Cost of Defects
- 3 Levels of Testing
- 4 The V-Model and STLC
- 5 Common Misconceptions
- 6 Wrap-Up**

A Practical Mental Model

When you hear: “This module passed code review and all its unit tests.”

Think: It has been *verified*. Has anyone *validated* that the specification itself is what the system needs?

When you hear: “We didn’t find any bugs in this release.”

Think: Our tests found no failures — not that no faults exist. What parts of the input space did we not exercise?

A Practical Mental Model, Continued

When you hear: “This defect is only in an edge case; it’s probably not a big deal.”

Think: Severity and priority are different questions. How bad *if* it happened, and separately, how urgent to fix?

When you hear: “We’ll test it thoroughly before the next release.”

Think: The cost of every defect we find today is lower than tomorrow. What can shift left?

Recap: Key Takeaways

Severity and priority are independent dimensions — assess a defect's impact and its urgency separately.

Cost grows with delay — the core argument for testing early and often.

Testing levels increase in scope: unit → integration → system → acceptance, mirrored by the V-model.

The STLC runs alongside development, not after it — acceptance tests can exist before code does.

The Big Picture: Both Lectures Together

Term	Meaning
Error / Fault / Failure	A mistake, the resulting flaw, and its observable effect
Verification / Validation	Built <i>right</i> ? vs. the <i>right</i> product?
Severity / Priority	Technical impact vs. fix urgency
Shift-left testing	Moving testing earlier to reduce cost
Unit / Integration / System / Acceptance	The four levels of testing, increasing in scope
STLC	The phases the testing process itself moves through

Key Idea

Testing builds justified confidence; it does not build proof.

Questions?

Thank you.

SE 413/513 — Software Testing
Grand Valley State University